Stacklyst

Processos de negócio, modelos UML e implementação

Laboratório de Engenharia de Software

Revisão de 6 de outubro de 2026

Escopo do sistema apresentado

52 casos de uso

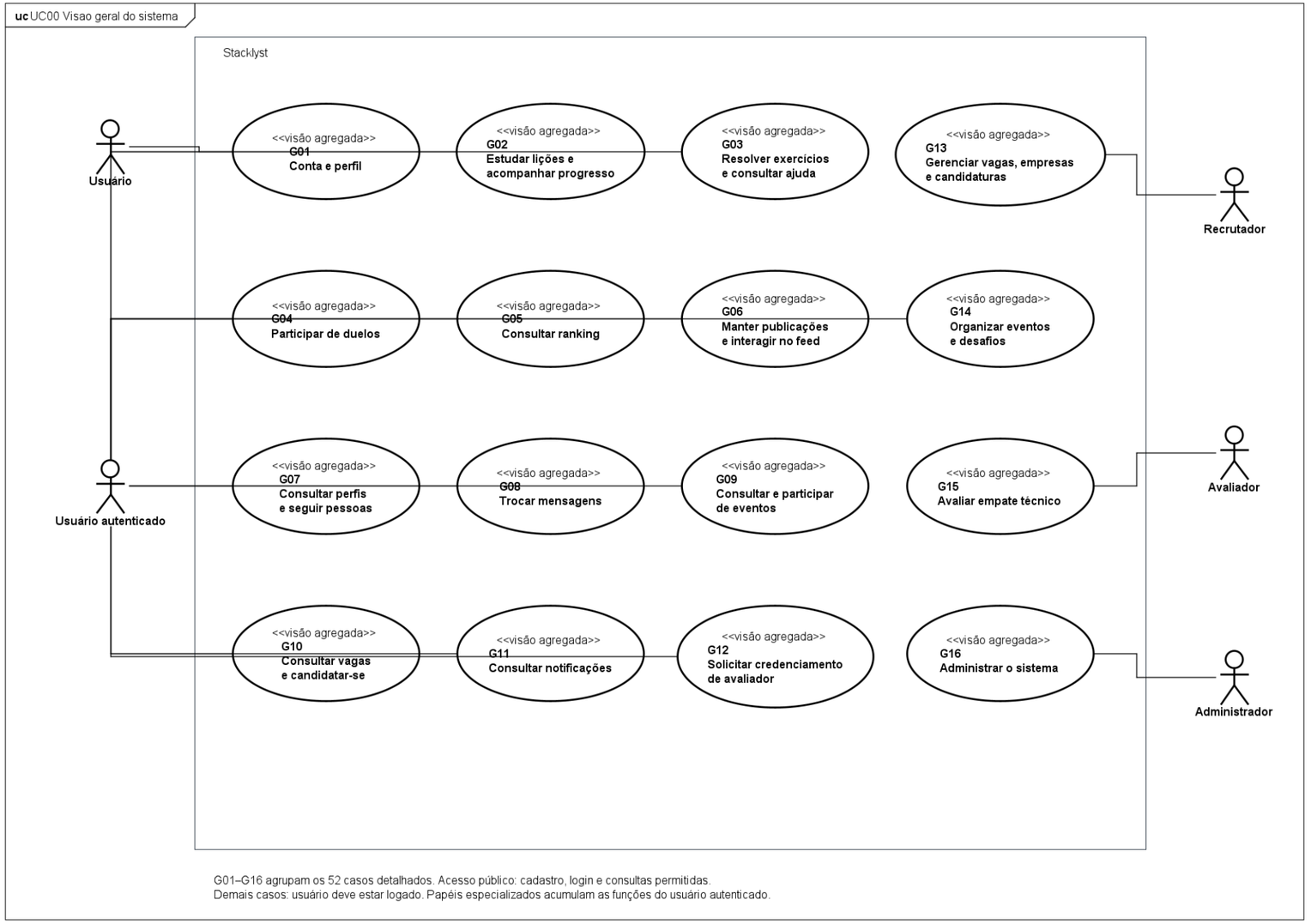
Estudo, exercícios, duelos, publicações, mensagens, eventos, vagas e administração.

Candidatura à vaga e inscrição no evento são as operações implementadas. A notificação da candidatura acompanha a atualização feita pelo recrutador.

Comunidades ficam fora do escopo desta entrega.

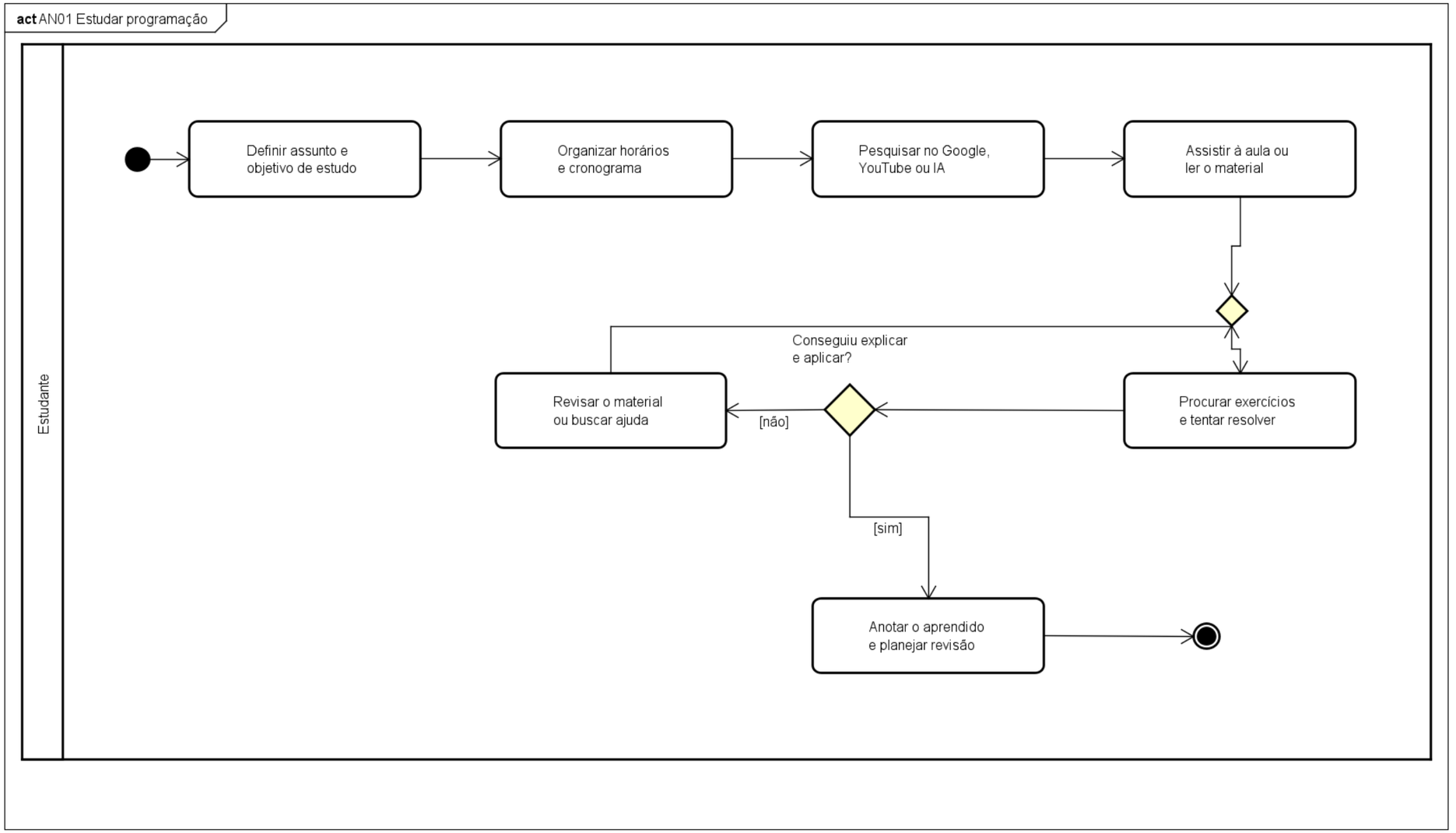
Visão geral de casos de uso

UC00 Agrupamentos das funcionalidades



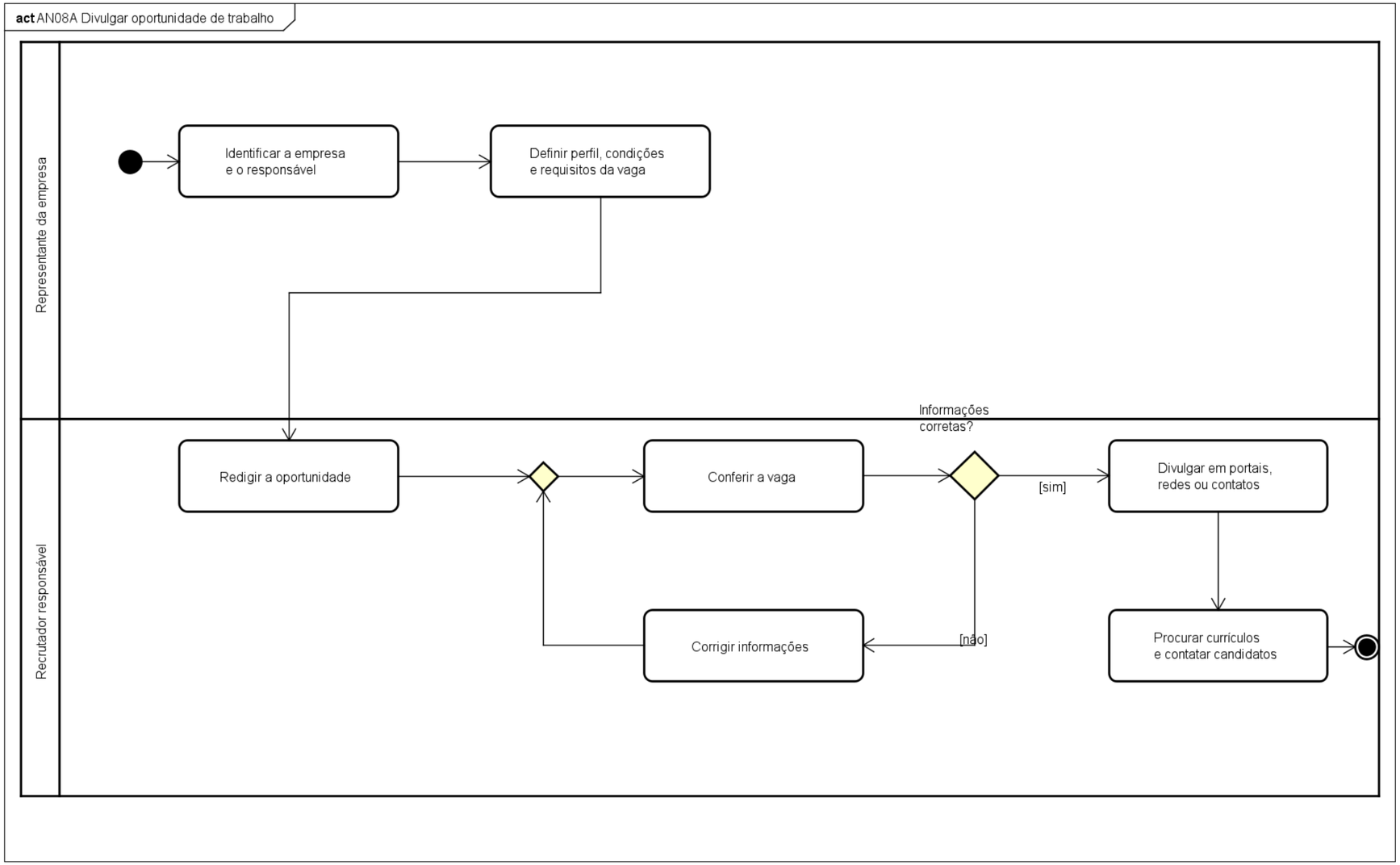
Estudar programação

AN01 Processo atual e responsabilidades



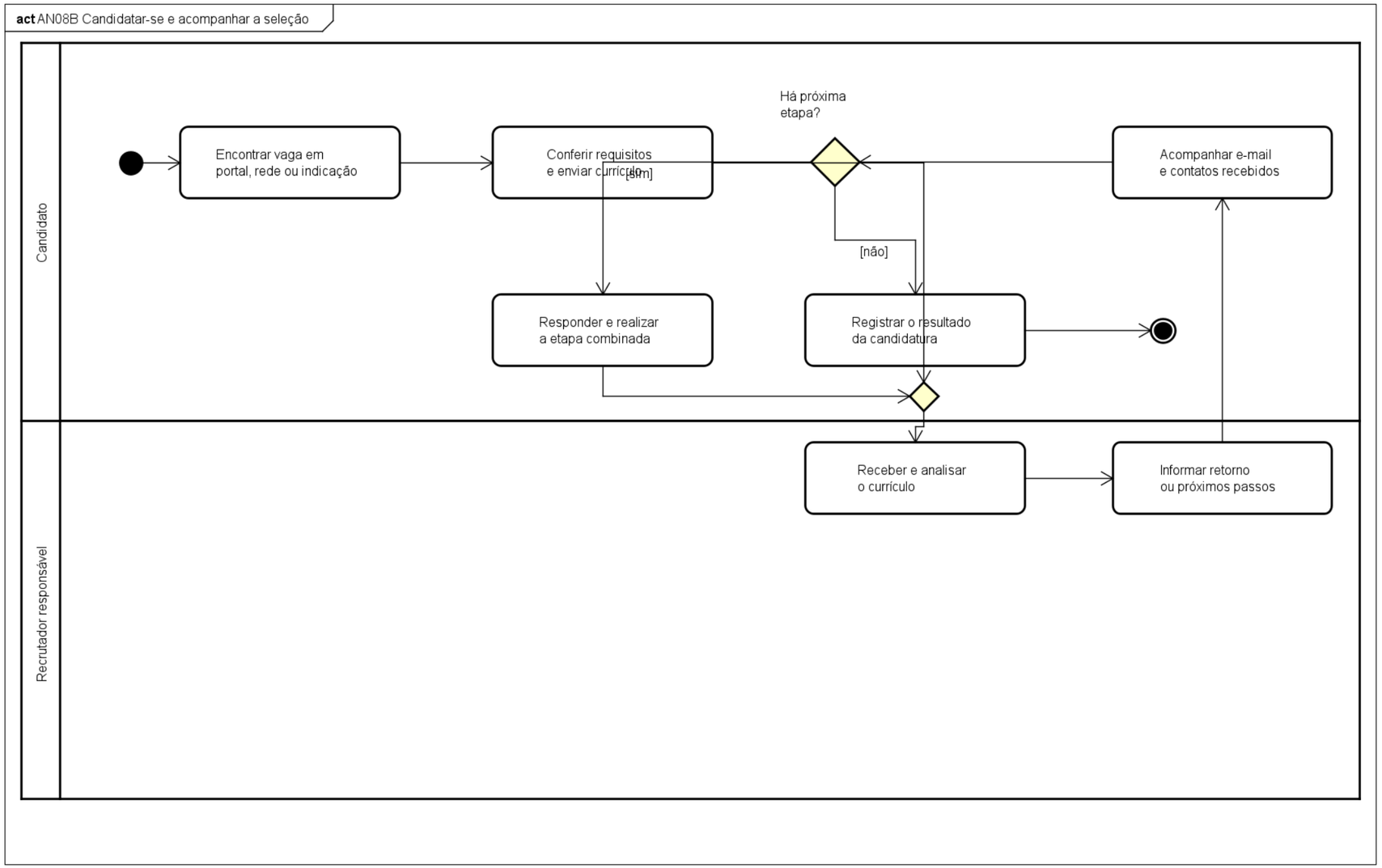
Divulgar oportunidade de trabalho

AN08A Processo atual e responsabilidades



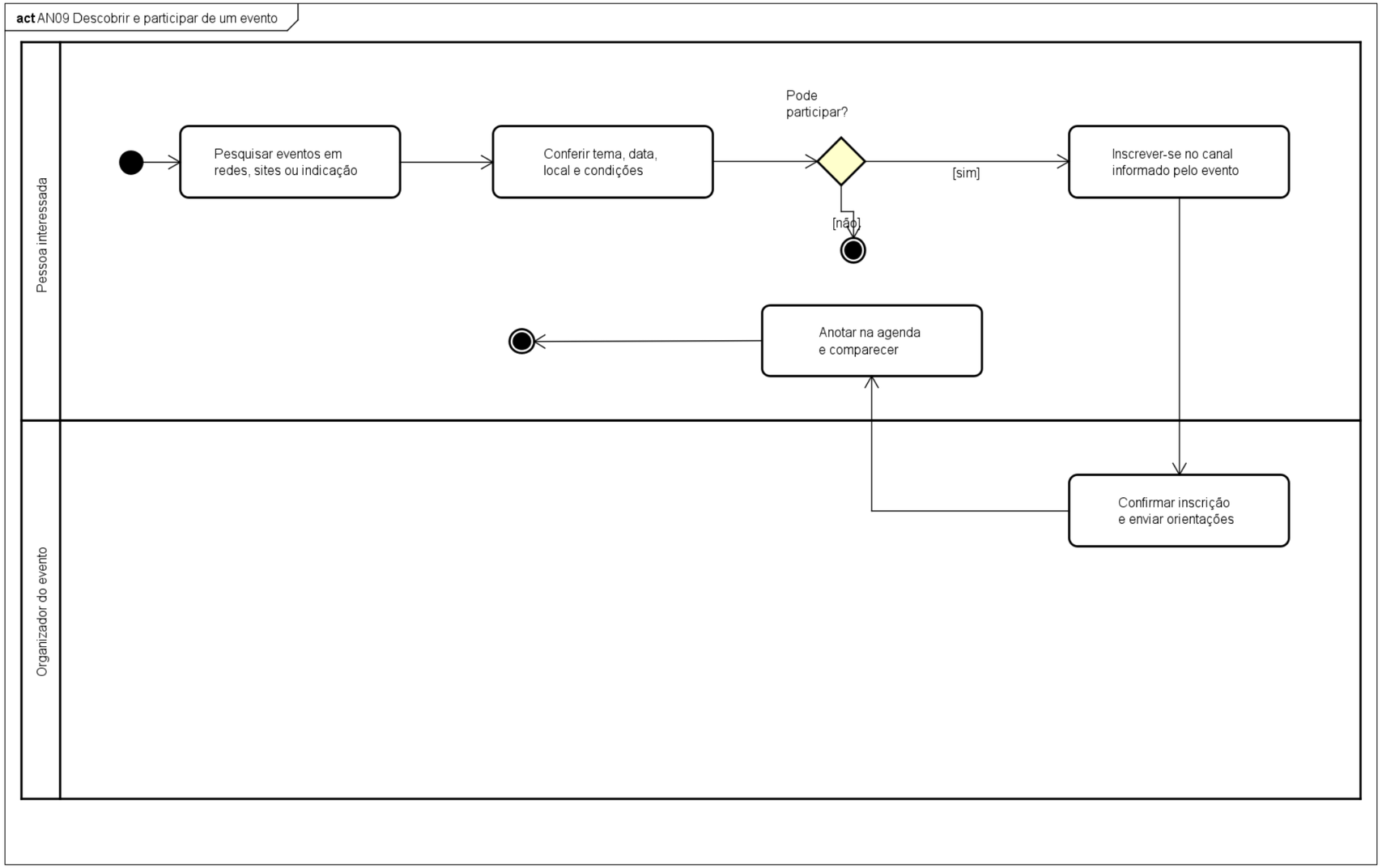
Candidatar-se e acompanhar a seleção

AN08B Processo atual e responsabilidades



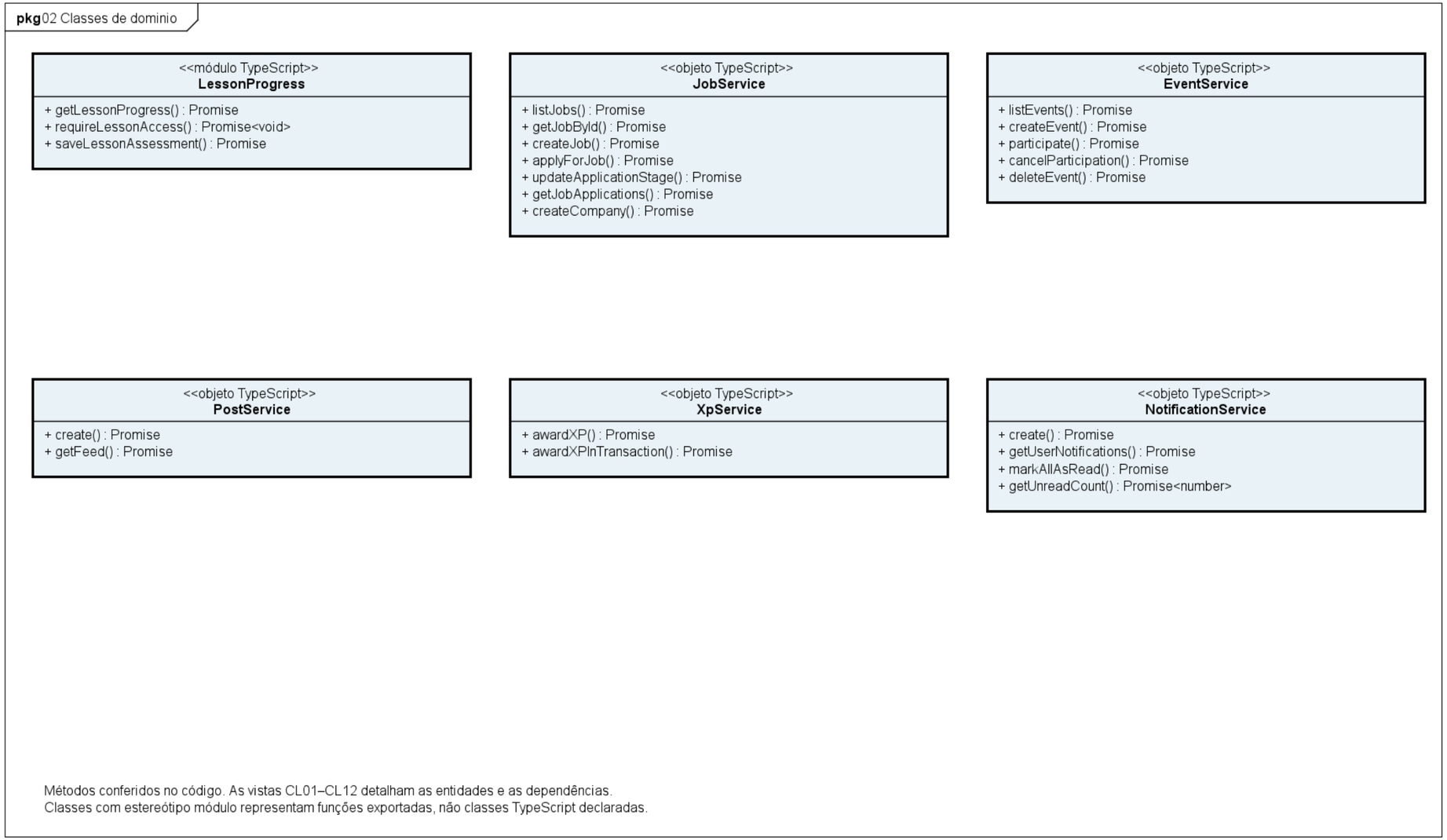
Descobrir e participar de um evento

AN09 Processo atual e responsabilidades



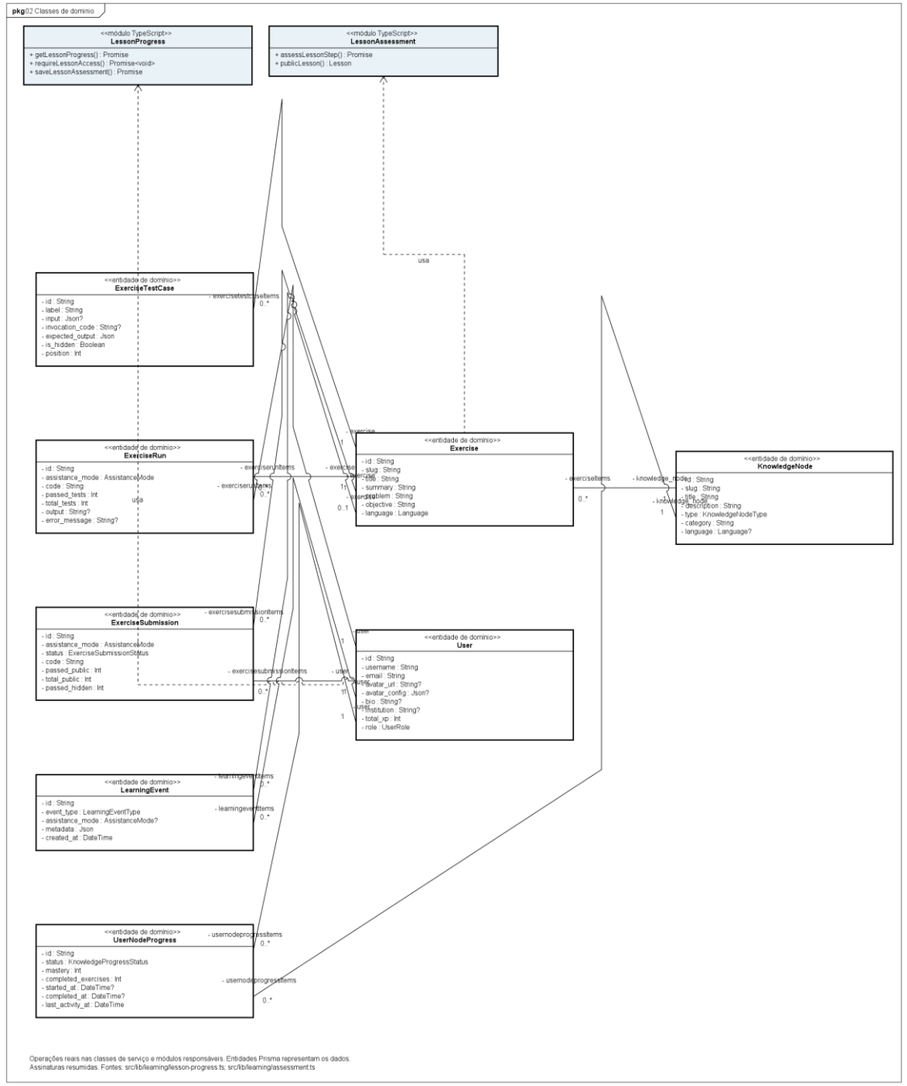
Classes e responsabilidades principais

CL00 Operações e dados sob responsabilidade



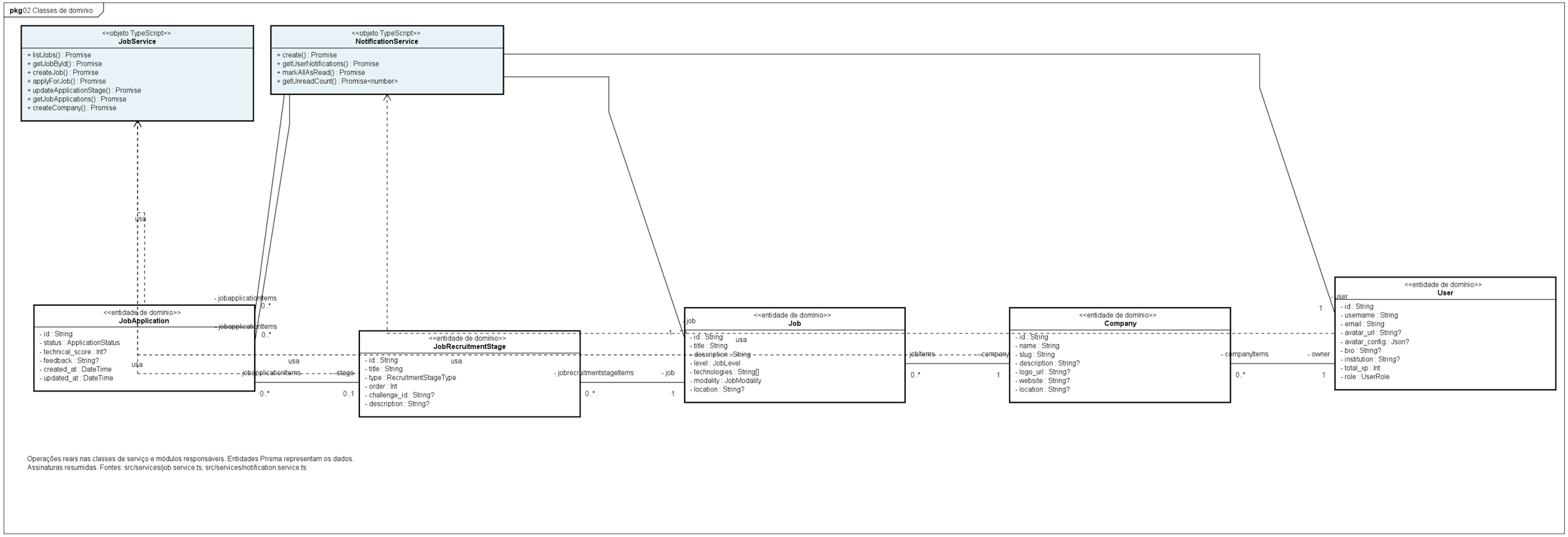
Pratica e progresso

CL03 Operações e dados sob responsabilidade



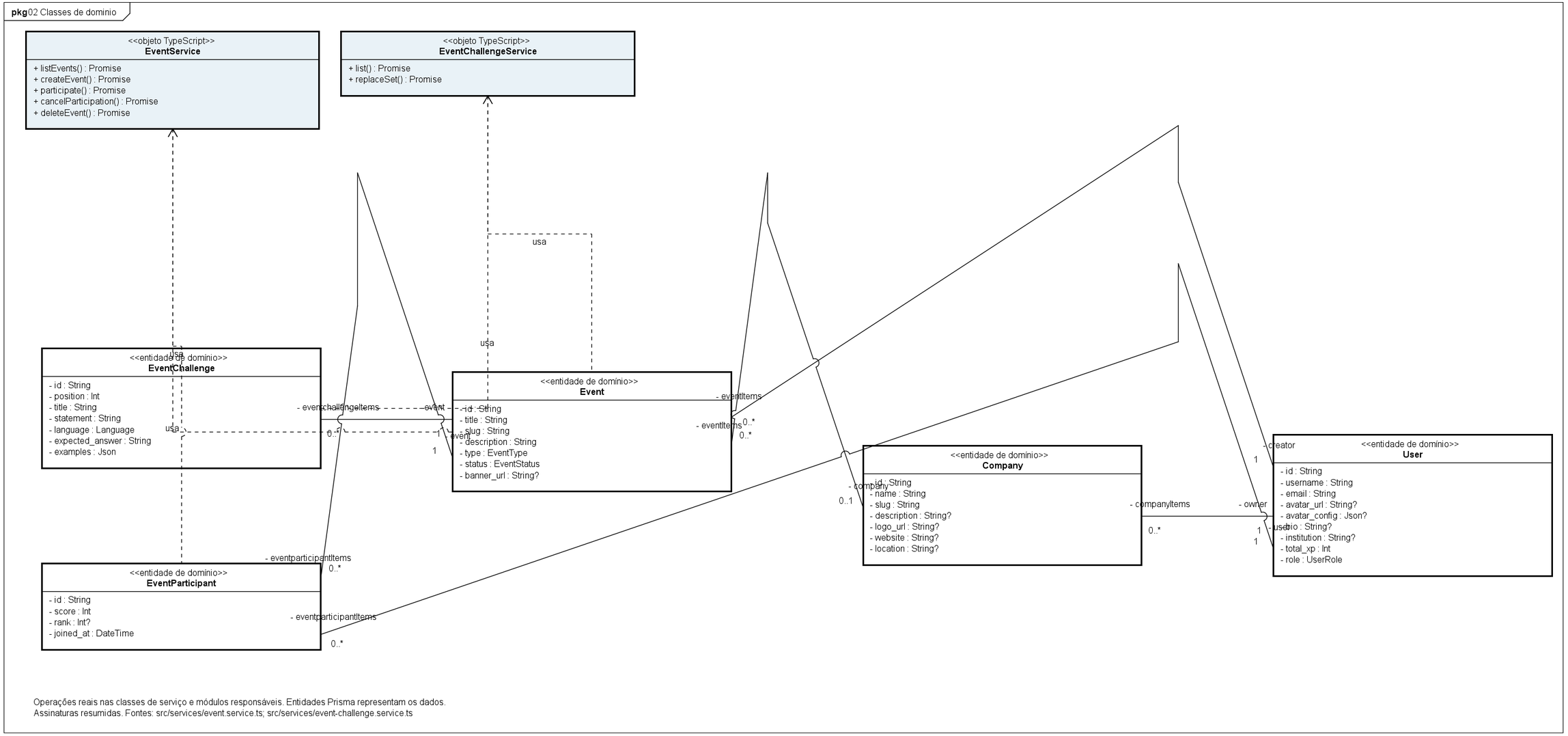
Vagas e empresas

CL08 Operações e dados sob responsabilidade



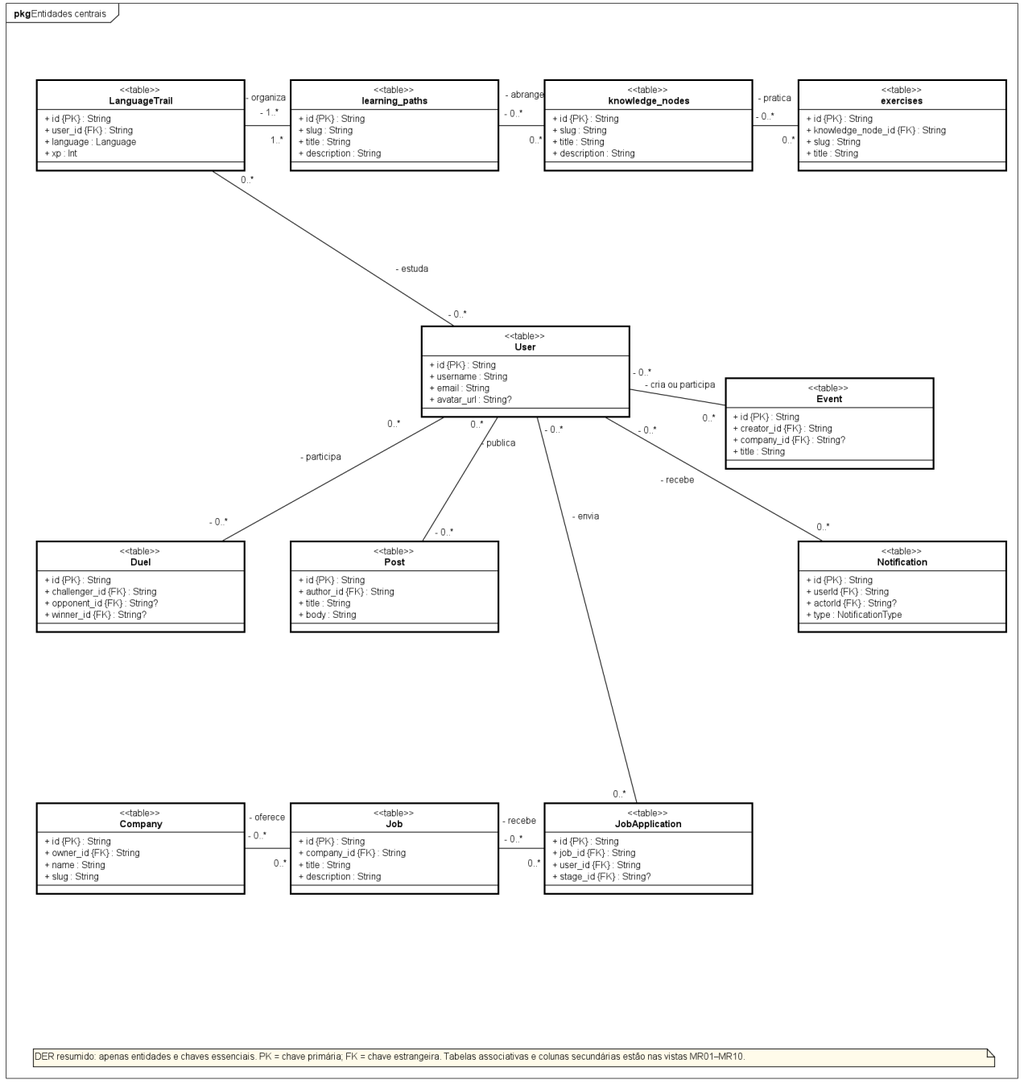
Eventos

CL09 Operações e dados sob responsabilidade



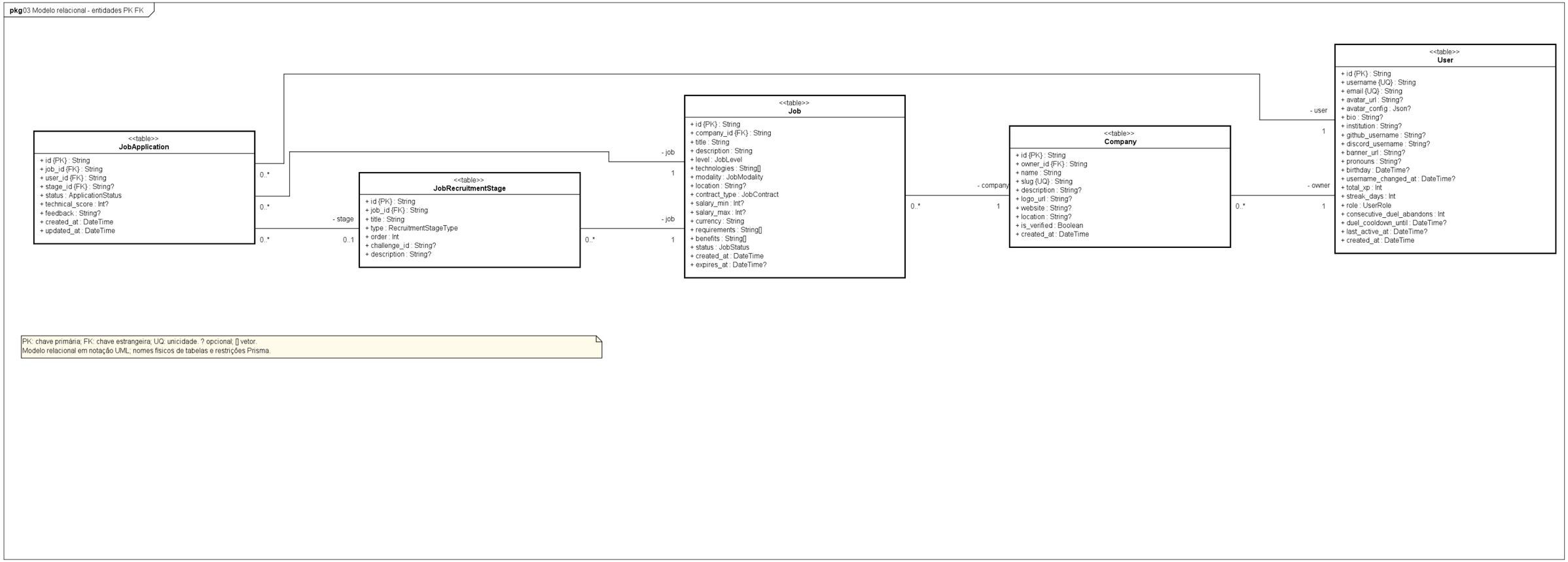
DER do núcleo do Stacklyst

MR00 Chaves e relacionamentos



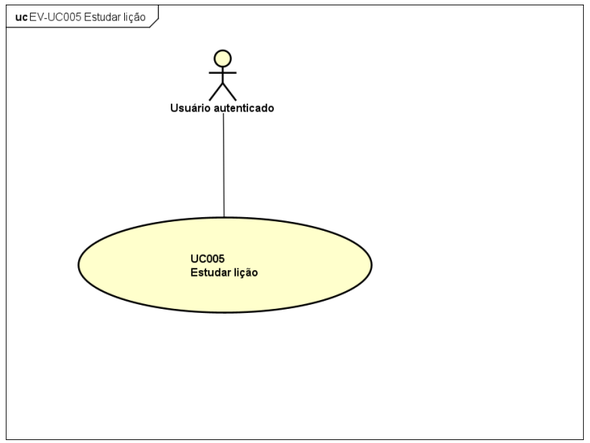
DER de vagas e candidaturas

MR08 Chaves e relacionamentos



Estudar lição

UC005 Caso de uso e código correspondente



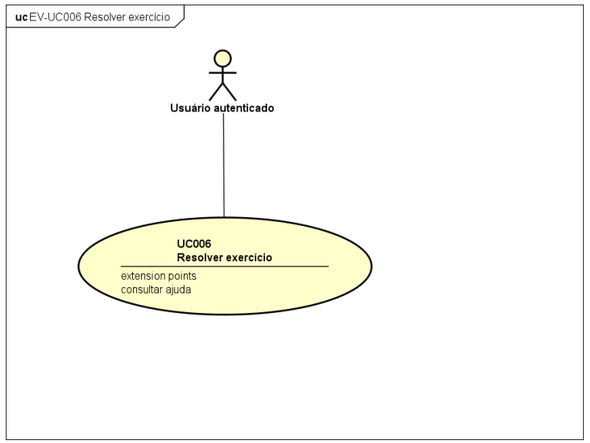
Carrega a lição, valida o acesso e entrega conteúdo e progresso à interface.

src/app/lesson/[lessonId]/page.tsx

```
59    if (!isExplicitWorkspace) {
60      const lesson = getLessonById(lessonId);
61      if (lesson) {
62        try {
63          await requireLessonAccess(user.id, lesson);
64        } catch (error) {
      Linhas intermediárias omitidas
76        const completedStepIds = await getLessonProgress(user.id, lesson);
77        return (
78          <LessonClient
79            lesson={publicLesson(lesson)}
80            completedStepIds={completedStepIds}
81            returnTo={returnTo}
82            user={{
83              id: user.id,
```

Resolver exercício

UC006 Caso de uso e código correspondente



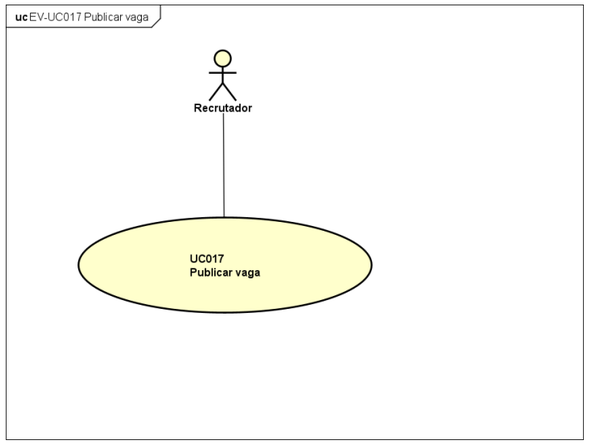
Avalia a resposta no servidor. Testar código não concede XP nem conclui a etapa.

src/app/api/lessons/[lessonId]/attempt/route.ts

```
21    const lesson = getLessonById(lessonId);
22    const step = lesson?.steps.find((s) => s.id === answer.stepId);
23    if (!lesson || !step) throw new AppError('LESSON_NOT_FOUND', 'Atividade não encontrada.', 404);
24    await requireLessonAccess(user.id, lesson);
25    const outcome = await assessLessonStep(lesson, step, answer);
26    if ('unavailable' in outcome && outcome.unavailable)
27      throw new AppError(
28        'RUNNER_UNAVAILABLE',
29        'Serviço de execução indisponível. Seu progresso não foi alterado; tente novamente.',
30        503
31      );
32    if (answer.action === 'run') return NextResponse.json({ ...outcome, xpEarned: 0 });
33    const progress = await saveLessonAssessment(
34      user.id,
35      lesson,
36      step,
37      outcome.isCorrect,
38      answer.selectedOption ?? 0
39    );
40    return NextResponse.json({ ...outcome, ...progress });
```

Publicar vaga

UC017 Caso de uso e código correspondente



Exige papel de recrutador ou administrador e vínculo autorizado com a empresa.

src/app/api/jobs/route.ts

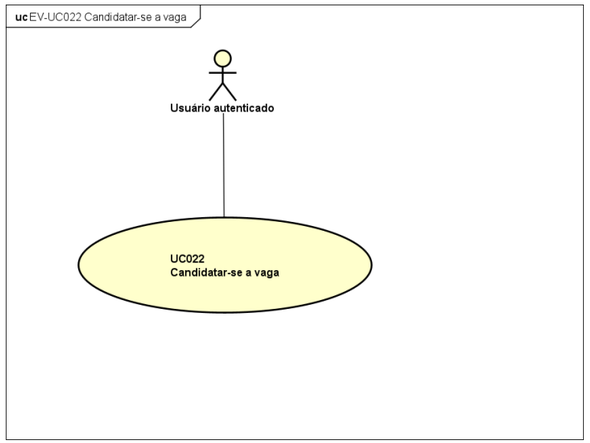
```
67  export const POST = apiHandler(async (req) => {
68    const user = await requireRole(['RECRUITER', 'ADMIN']);
69    const body = await req.json();
70    const parsed = createJobSchema.parse(body);
71    await requireCompanyAccess(parsed.company_id, user);
72
73    const job = await JobService.createJob({
74      companyId: parsed.company_id,
75      title: parsed.title,
76      description: parsed.description,
77      level: parsed.level,
78      technologies: parsed.technologies,
79      modality: parsed.modality,
80      location: parsed.location,
```

Linhas intermediárias omitidas

```
89    return NextResponse.json({
90      success: true,
91      message: 'Vaga publicada com sucesso!',
92      job,
93    });
94  });
```

Candidatar-se a vaga

UC022 Caso de uso e código correspondente



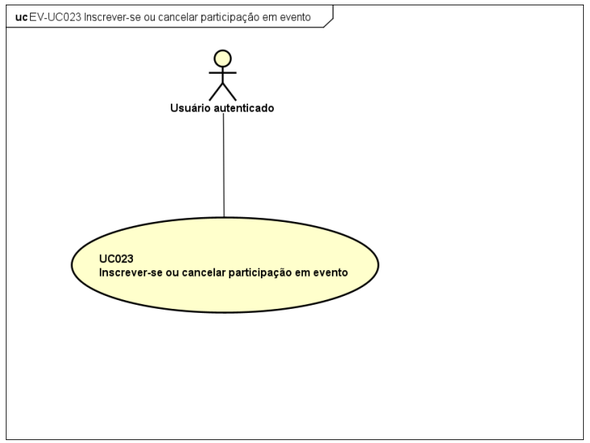
Verifica a vaga aberta e cria uma candidatura vinculada ao usuário e à etapa inicial.

src/services/job.service.ts

```
269         include: {
270           stages: { orderBy: { order: 'asc' } },
271           company: true,
272         },
273       });
274
275       if (!job || job.status !== 'OPEN') {
276         throw new Error('Esta vaga não está mais aceitando inscrições.');
277       }
278
279       const firstStage = job.stages[0];
280
281       const application = await prisma.jobApplication.create({
282         data: {
283           job_id: jobId,
284           user_id: userId,
285           stage_id: firstStage?.id,
286           status: 'APPLIED',
287         },
288         include: {
289           stage: true,
290           job: { include: { company: true } },
291         },
```

Inscrever-se ou cancelar participação em evento

UC023 Caso de uso e código correspondente



src/app/api/events/[id]/participate/route.ts

```
6  export const POST = apiHandler(async (_req, { params }) => {
7    const user = await requireAuth();
8    const { id: eventId } = await params;
9
10   const participant = await EventService.participate(user.id, eventId);
11
12   return NextResponse.json({
13     success: true,
14     message: 'Inscrição no evento confirmada!',
15     participant,
16   });
```

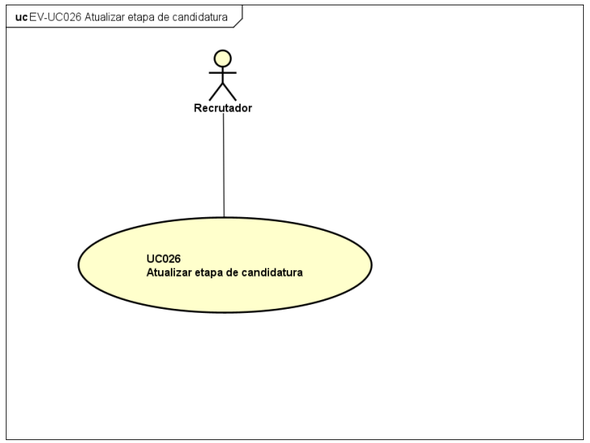
Linhas intermediárias omitidas

```
19 export const DELETE = apiHandler(async (_req, { params }) => {
20   const user = await requireAuth();
21   const { id: eventId } = await params;
22
23   await EventService.cancelParticipation(user.id, eventId);
24
25   return NextResponse.json({
26     success: true,
27     message: 'Inscrição cancelada com sucesso.',
28   });
29 });
```

POST confirma a inscrição.
DELETE cancela a participação do usuário autenticado.

Atualizar etapa de candidatura

UC026 Caso de uso e código correspondente



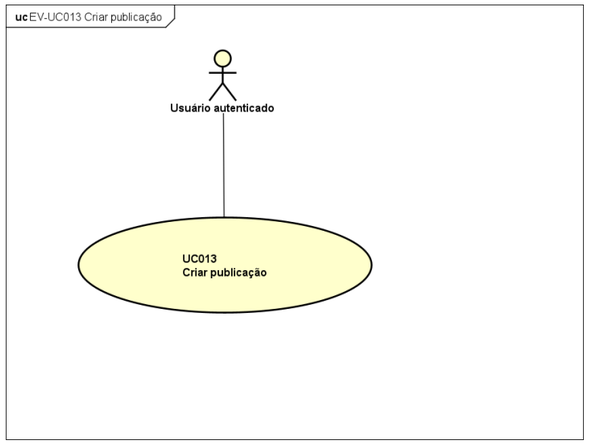
Confere a empresa, a candidatura e a etapa antes de atualizar o processo.

src/app/api/jobs/[id]/route.ts

```
30  export const PATCH = apiHandler(async (req, { params }) => {
31    const user = await requireRole(['RECRUITER', 'ADMIN']);
32    const { id } = await params;
33    const job = await prisma.job.findUniqueOrThrow({ where: { id } });
34    await requireCompanyAccess(job.company_id, user);
35    const body = await req.json();
36    const parsed = updateStageSchema.parse(body);
37    const application = await prisma.jobApplication.findFirst({
38      where: { id: parsed.application_id, job_id: id },
39    });
40    if (!application)
41      return NextResponse.json({ error: 'Candidatura não encontrada nesta vaga.' }, { status: 404 });
42    if (
43      parsed.stage_id &&
44      !(await prisma.jobRecruitmentStage.findFirst({ where: { id: parsed.stage_id, job_id: id } }))
45    )
46      return NextResponse.json({ error: 'Etapa inválida para esta vaga.' }, { status: 400 });
      Linhas intermediárias omitidas
48    const updatedApplication = await JobService.updateApplicationStage({
49      applicationId: parsed.application_id,
50      stageId: parsed.stage_id,
51      status: parsed.status,
52      feedback: parsed.feedback,
53      technicalScore: parsed.technical_score,
54    });
```

Criar publicação

UC013 Caso de uso e código correspondente



A criação grava a publicação e concede XP na mesma transação.

src/services/post.service.ts

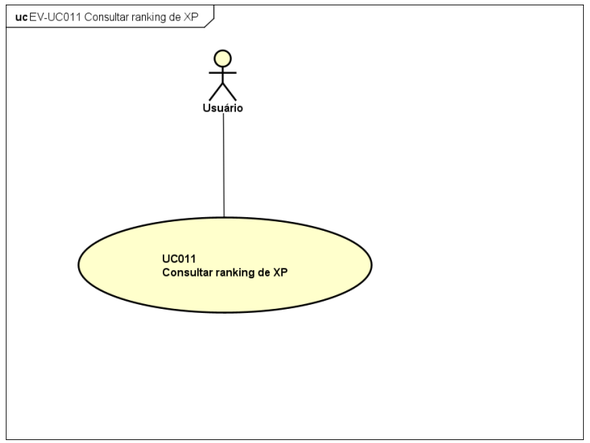
```
25      const result = await prisma.$transaction(async (tx) => {
26        // 1. Create Post
27        const post = await tx.post.create({
28          data: {
29            author_id: userId,
30            title,
31            body,
32            language: language || null,
33            code_snippet: code || null,
34            image_url: image_url || null,
35            client_id: clientId || null,
36          },
37        });
38
```

Linhas intermediárias omitidas

```
40        const xpAmount = language ? 10 : 5;
41        const beforeUser = await tx.user.findUnique({
42          where: { id: userId },
43          select: { total_xp: true },
44        });
45
46        const xpResult = await XpService.awardXPInTransaction(tx, userId, language, xpAmount);
```

Consultar ranking de XP

UC011 Caso de uso e código correspondente



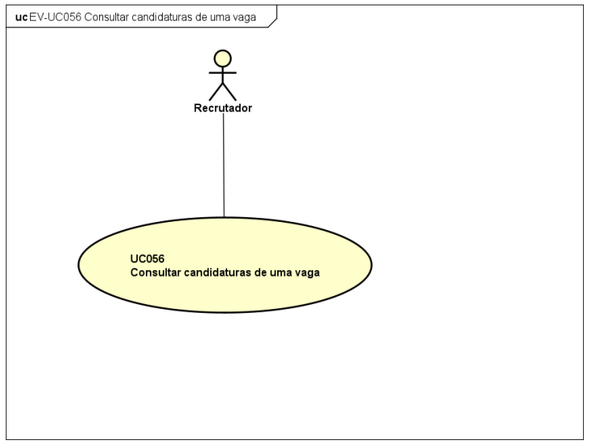
O servidor consulta e ordena os usuários. O caso de uso é consulta, sem manutenção manual do ranking.

src/app/api/leaderboard/route.ts

```
31        // Leaderboard global baseado no total_xp do usuário
32        const leaders = await prisma.user.findMany({
33          orderBy: XP_RANK_ORDER,
34          take: 50,
35          select: {
36            username: true,
37            avatar_url: true,
38            total_xp: true,
39            streak_days: true,
40            last_active_at: true,
41            created_at: true,
42          },
43        });
44
45        const formatted = leaders.map((leader, index) => ({
46          rank: index + 1,
47          username: leader.username,
48          avatar_url: leader.avatar_url,
49          xp: leader.total_xp,
50          level: calculateLevel(leader.total_xp).level,
51          streak: getEffectiveStreak(leader.streak_days ?? 0, leader.last_active_at),
52          created_at: leader.created_at ? leader.created_at.toISOString() : null,
53        }));
54
55        return NextResponse.json(formatted);
```

Consultar candidaturas de uma vaga

UC056 Caso de uso e código correspondente



src/app/api/jobs/[id]/applications/route.ts

```ts
import { NextResponse } from 'next/server';
import { apiHandler } from '@/lib/api-handler';
import { requireRole } from '@/lib/auth';
import { JobService } from '@/services/job.service';

export const GET = apiHandler(async (_req, { params }) => {
  const user = await requireRole(['RECRUITER', 'ADMIN']);
  const { id: jobId } = await params;

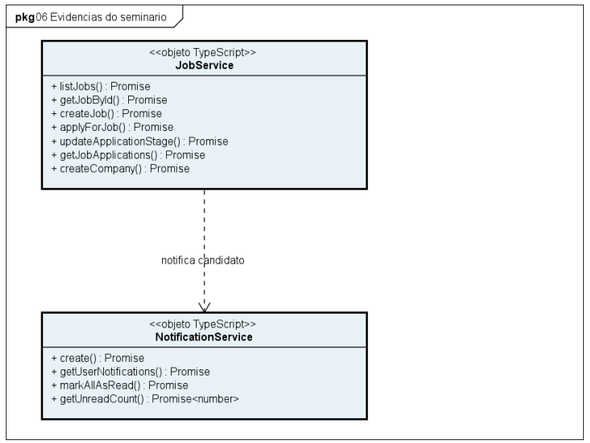
  const applications = await JobService.getJobApplications(jobId, user.id, user.role);

  return NextResponse.json(applications);
});
```

A rota exige papel autorizado. JobService também valida a responsabilidade pela empresa.

Notificação automática da candidatura

Comportamento interno ligado a UC026



A atualização avisa o candidato associado à candidatura. O código não envia um alerta geral por intenção de vaga.

src/services/job.service.ts

```
309
310     const application = await prisma.jobApplication.update({
311       where: { id: applicationId },
312       data: {
313         stage_id: stageId,
314         status,
315         feedback,
316         technical_score: technicalScore,
317       },
318       include: {
319         job: { include: { company: true } },
320         user: true,
321       },
322     });
323
324     try {
325       await NotificationService.create({
326         userId: application.user_id,
327         type: 'JOB_APPLICATION_UPDATE',
328         resourceId: application.job_id,
329         resourceType: 'JOB',
330       });
331     } catch (err) {
332       logger.error('Failed to notify job application update', { error: String(err) });
333     }
```

Relação entre processo e implementação

Processo de negócio

A pessoa define o objetivo, pesquisa, pratica e acompanha o resultado.

Comportamento do sistema

O caso de uso identifica a interação. O serviço executa a operação e registra o resultado.

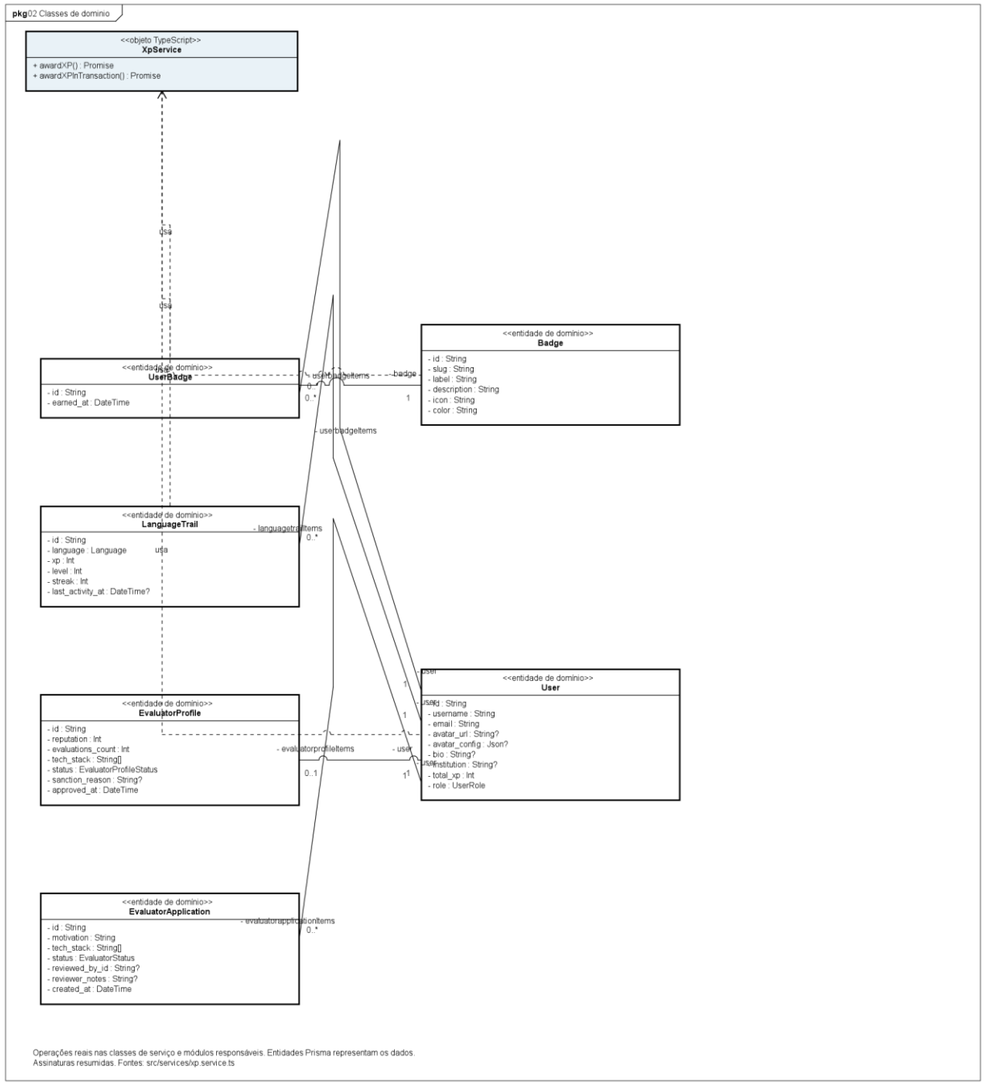
Os diagramas e as capturas usam os mesmos nomes e referências.

Classes por domínio e processos complementares de aprendizagem

Material de apoio para discussão

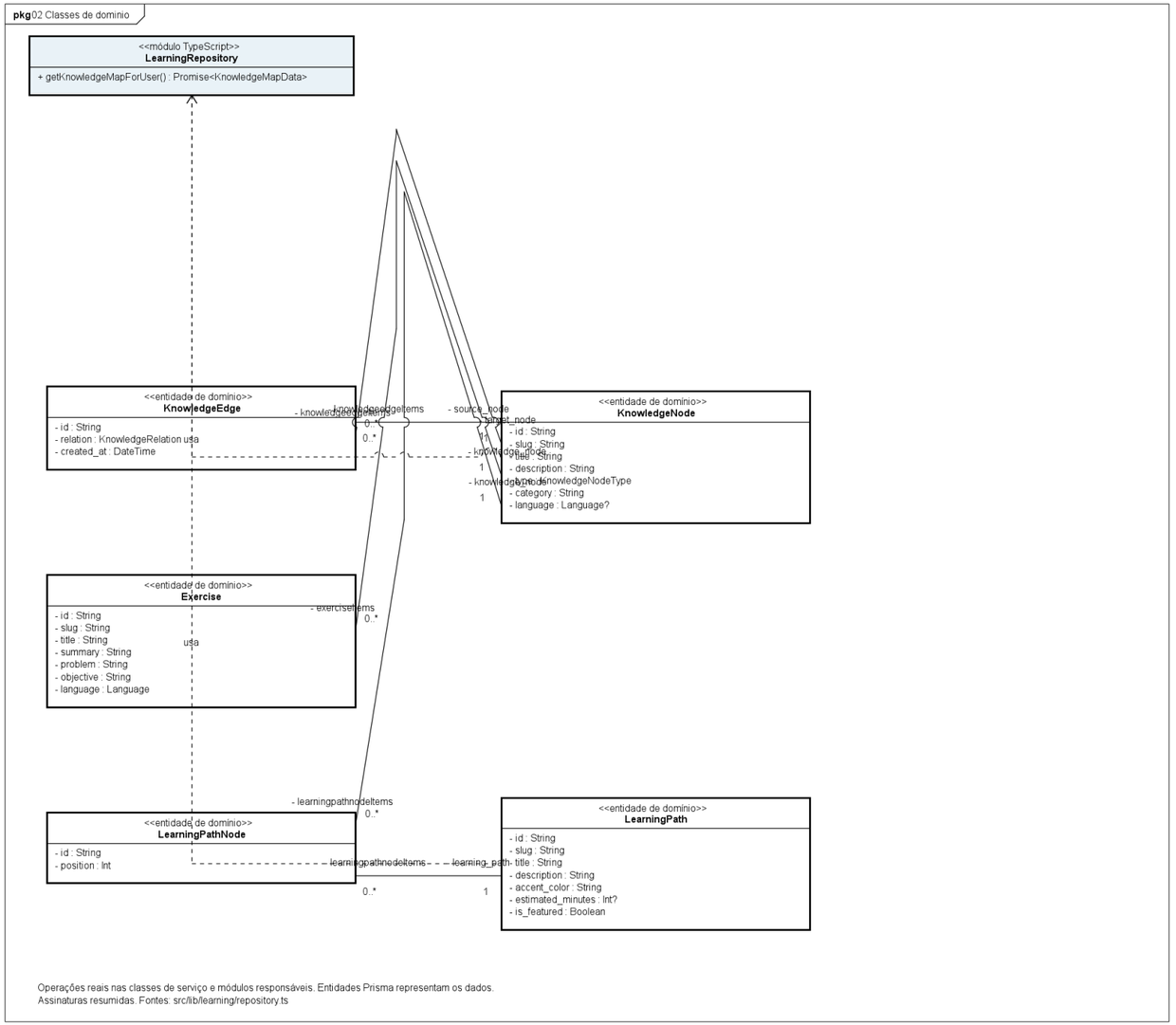
Identidade e gamificacao

CL01 Operações e dados sob responsabilidade



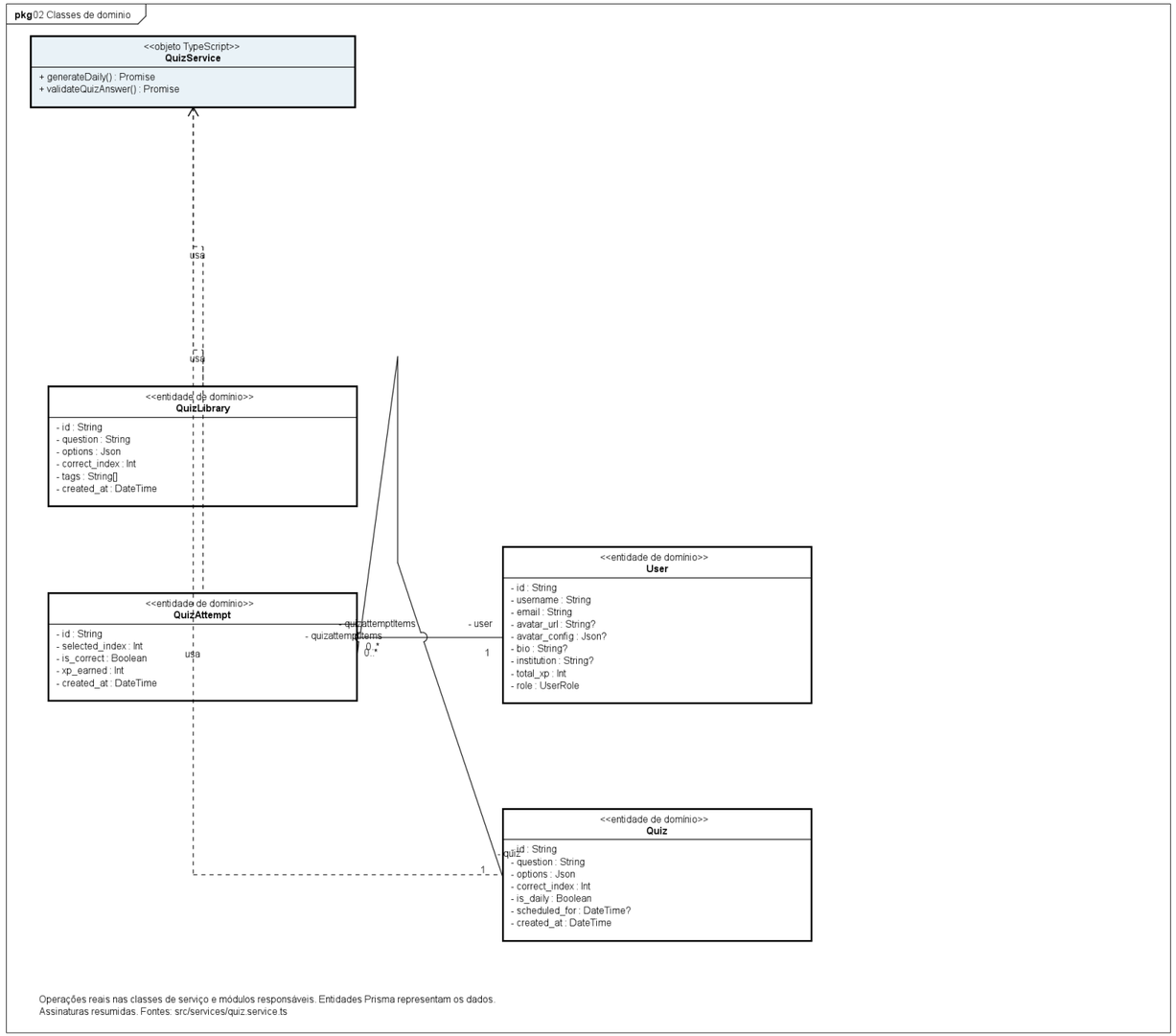
Grafo de conhecimento

CL02 Operações e dados sob responsabilidade



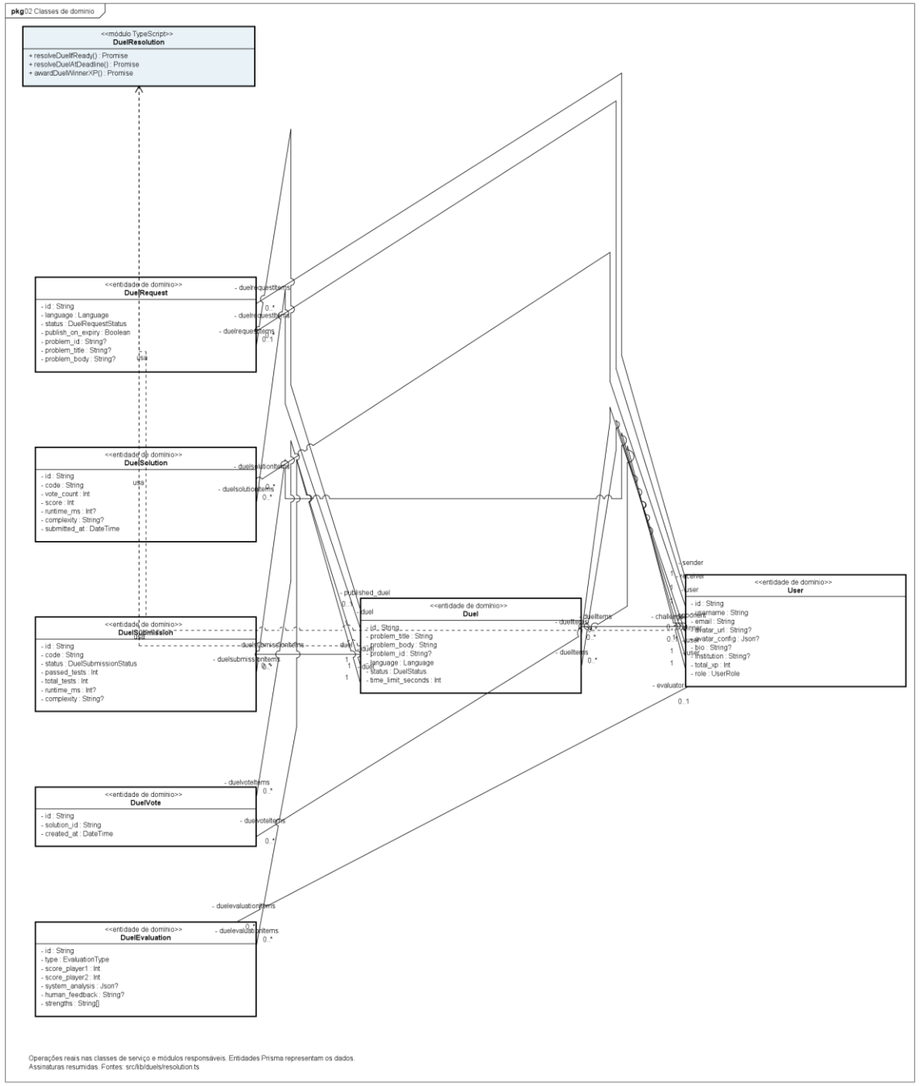
Quiz e historico

CL04 Operações e dados sob responsabilidade



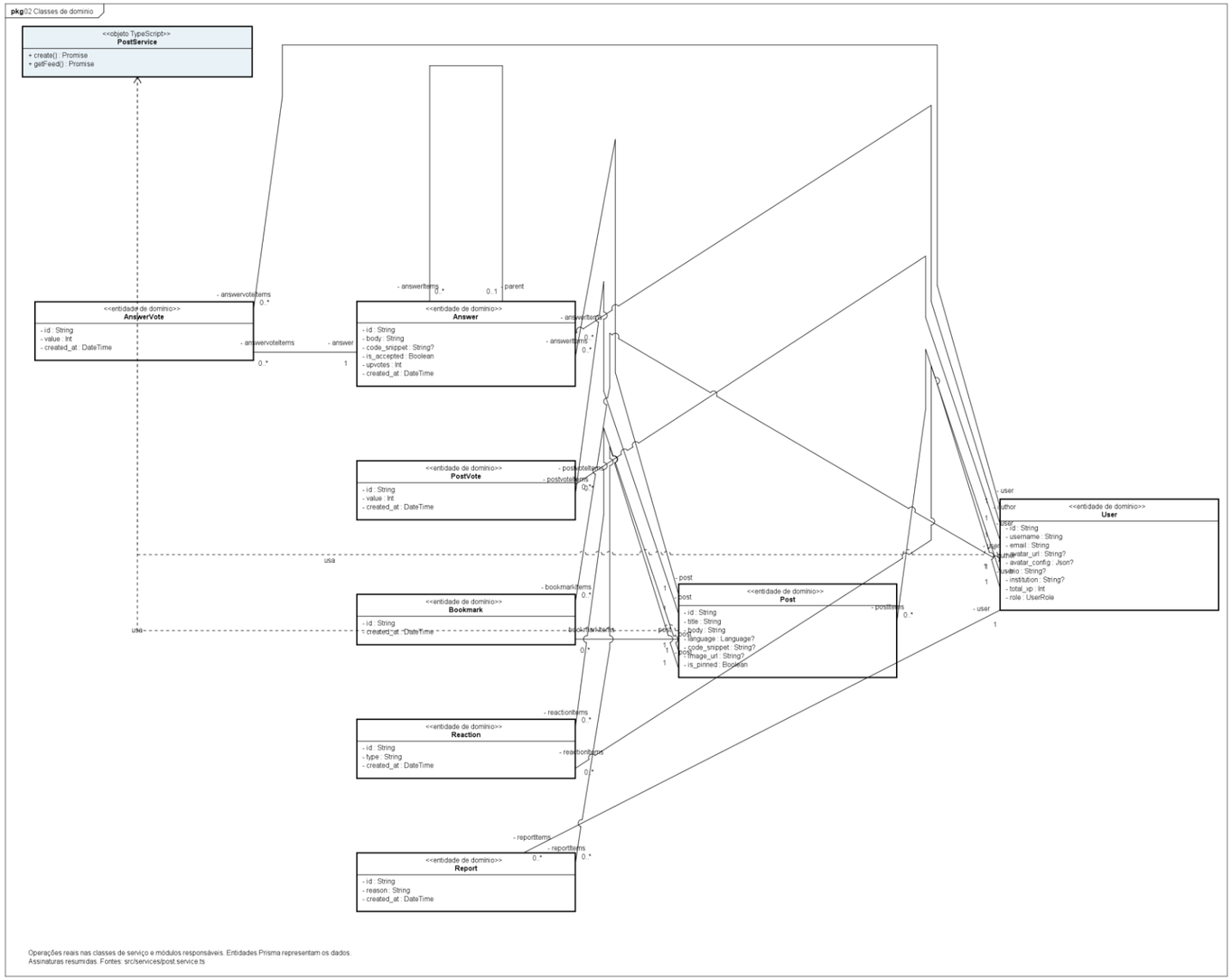
Duelos

CL05 Operações e dados sob responsabilidade



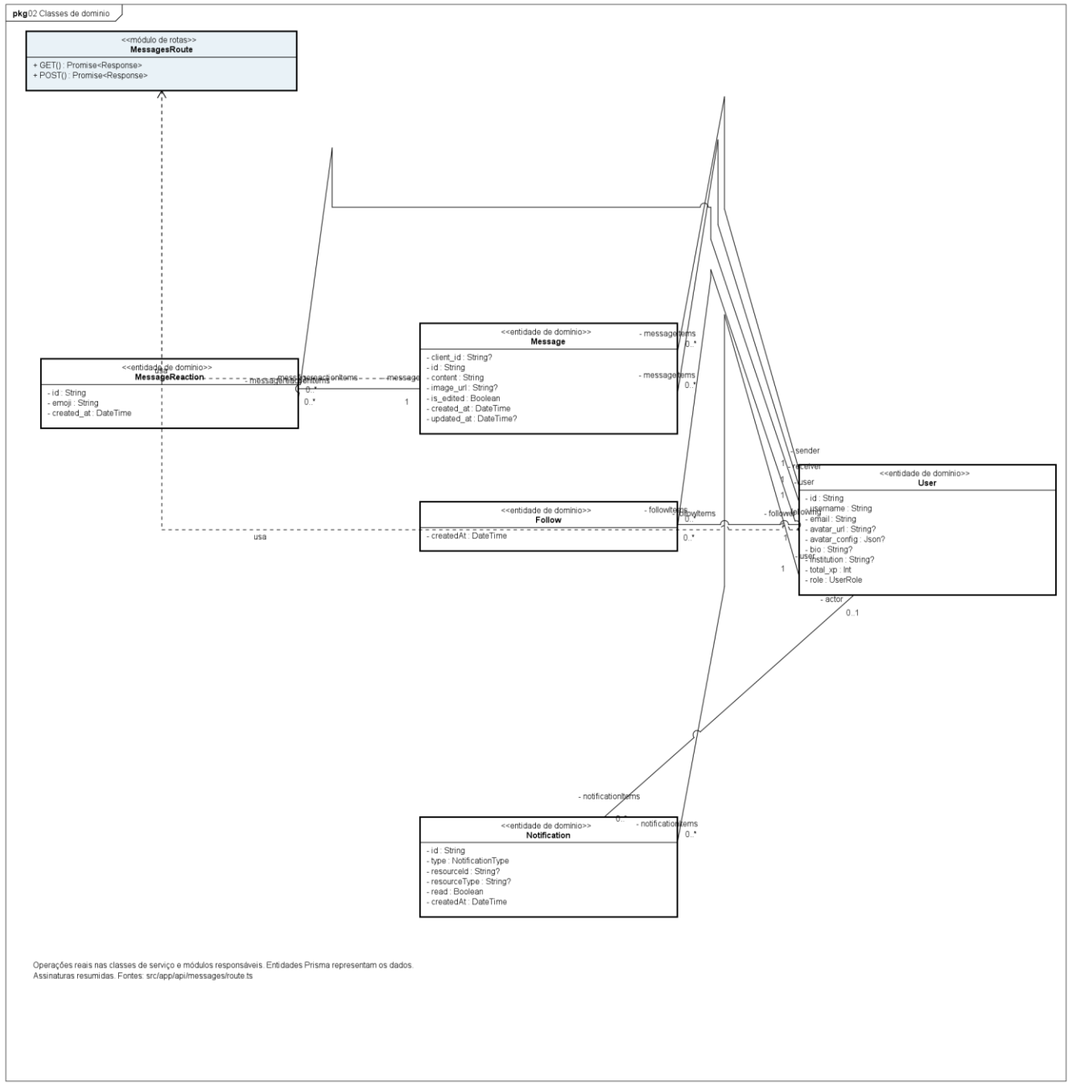
Publicacoes e moderacao

CL06 Operações e dados sob responsabilidade



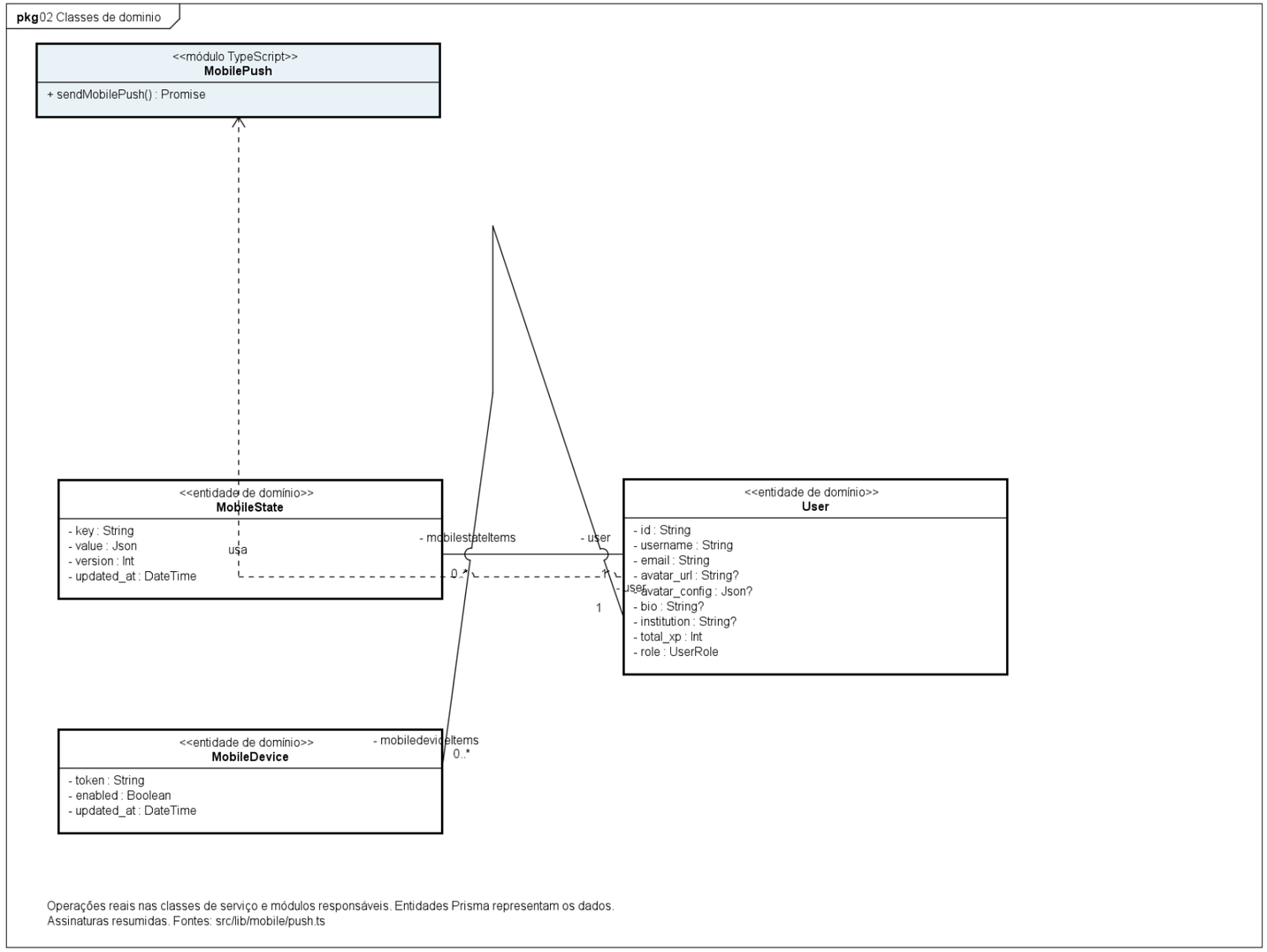
Mensagens privadas

CL07 Operações e dados sob responsabilidade



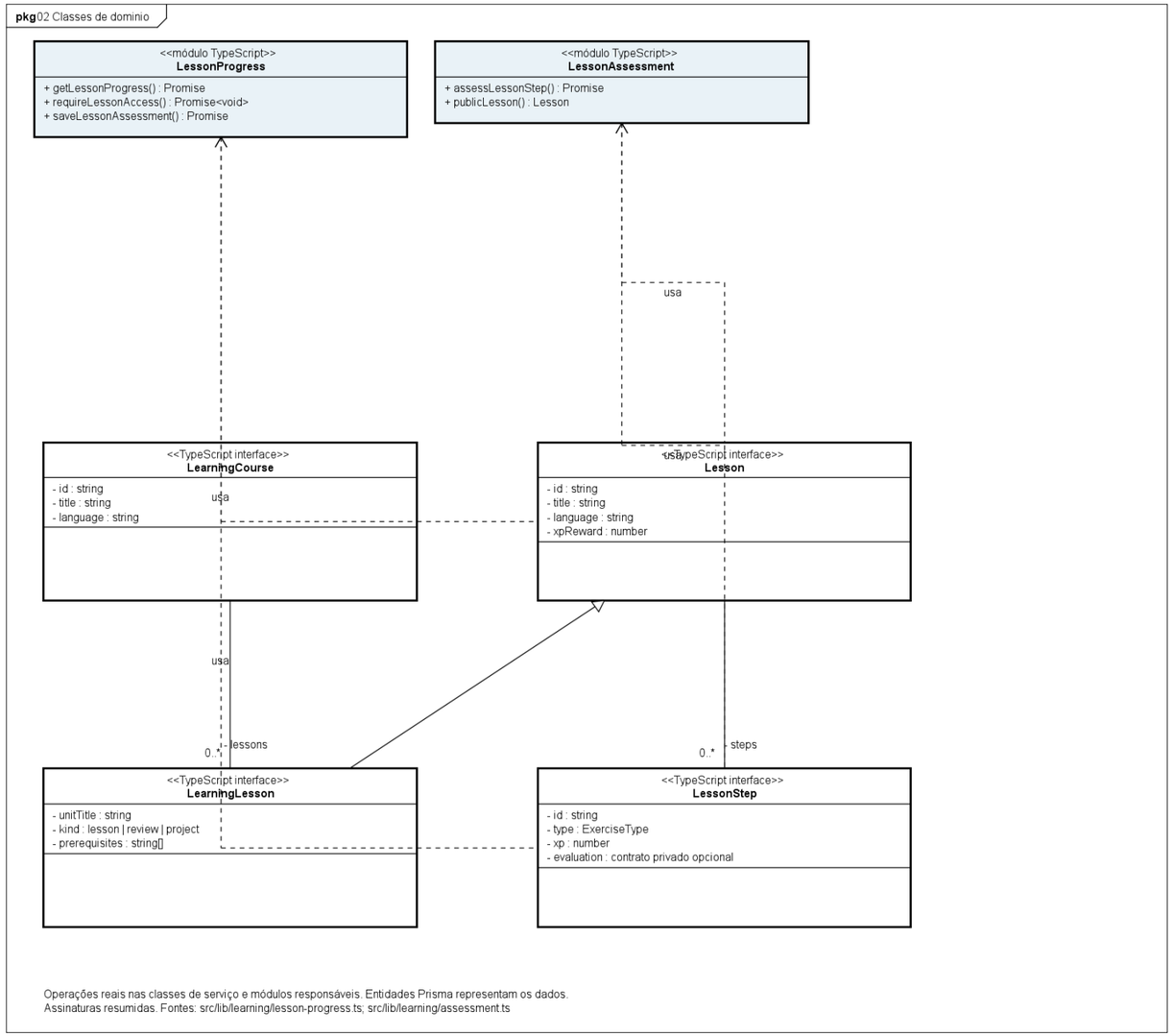
Suporte mobile

CL10 Operações e dados sob responsabilidade



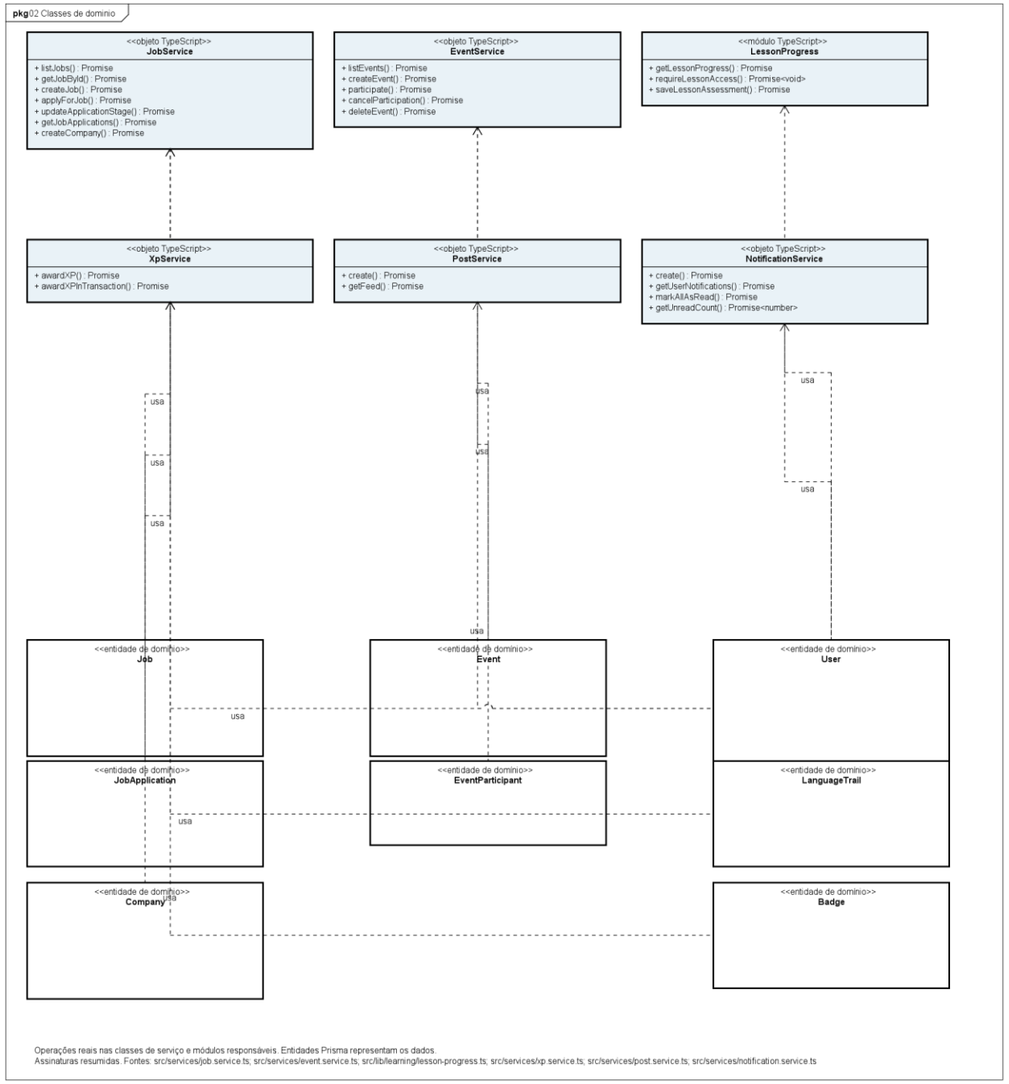
Catalogo e contratos TypeScript

CL11 Operações e dados sob responsabilidade



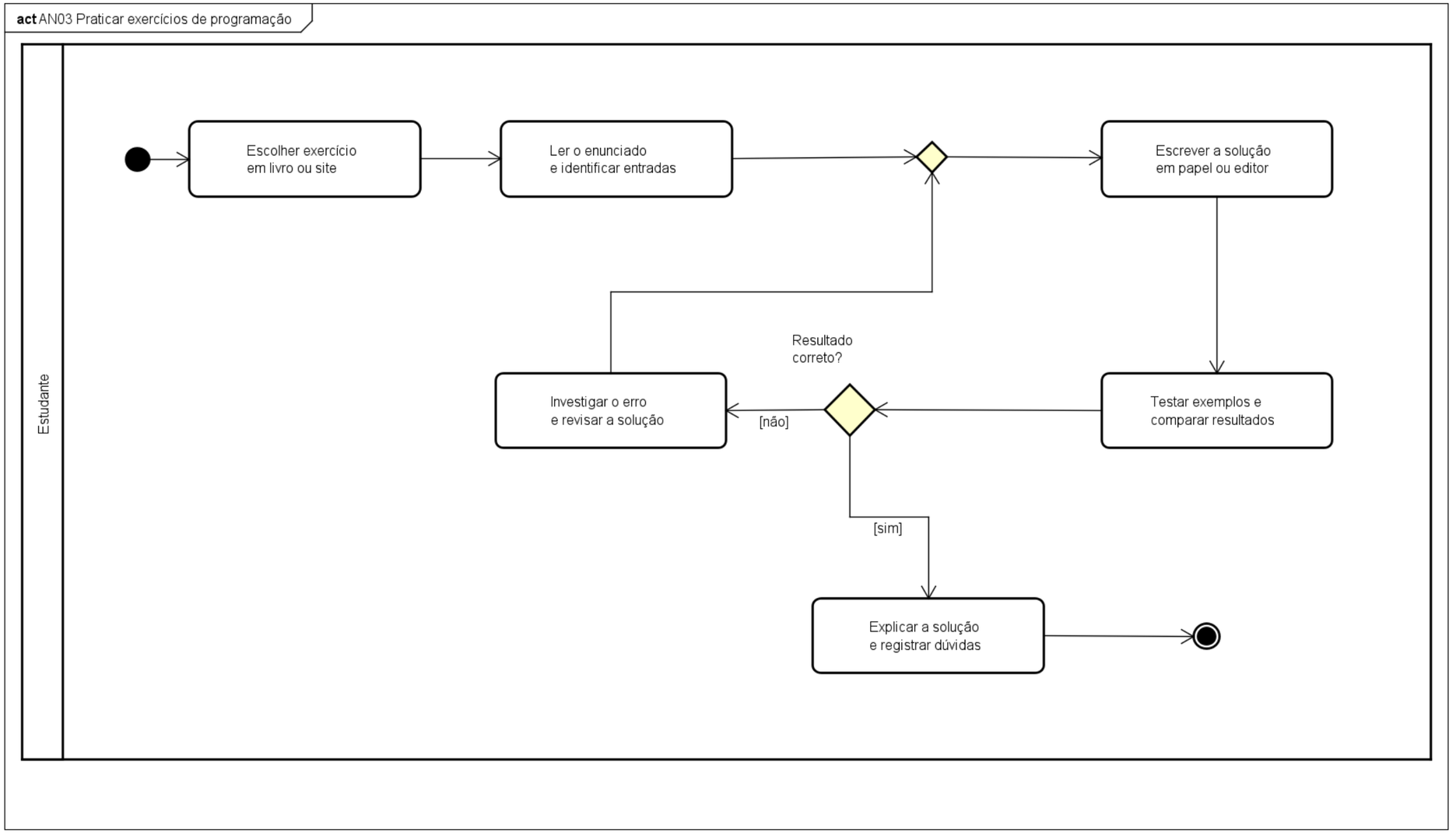
Servicos e operacoes reais

CL12 Operações e dados sob responsabilidade



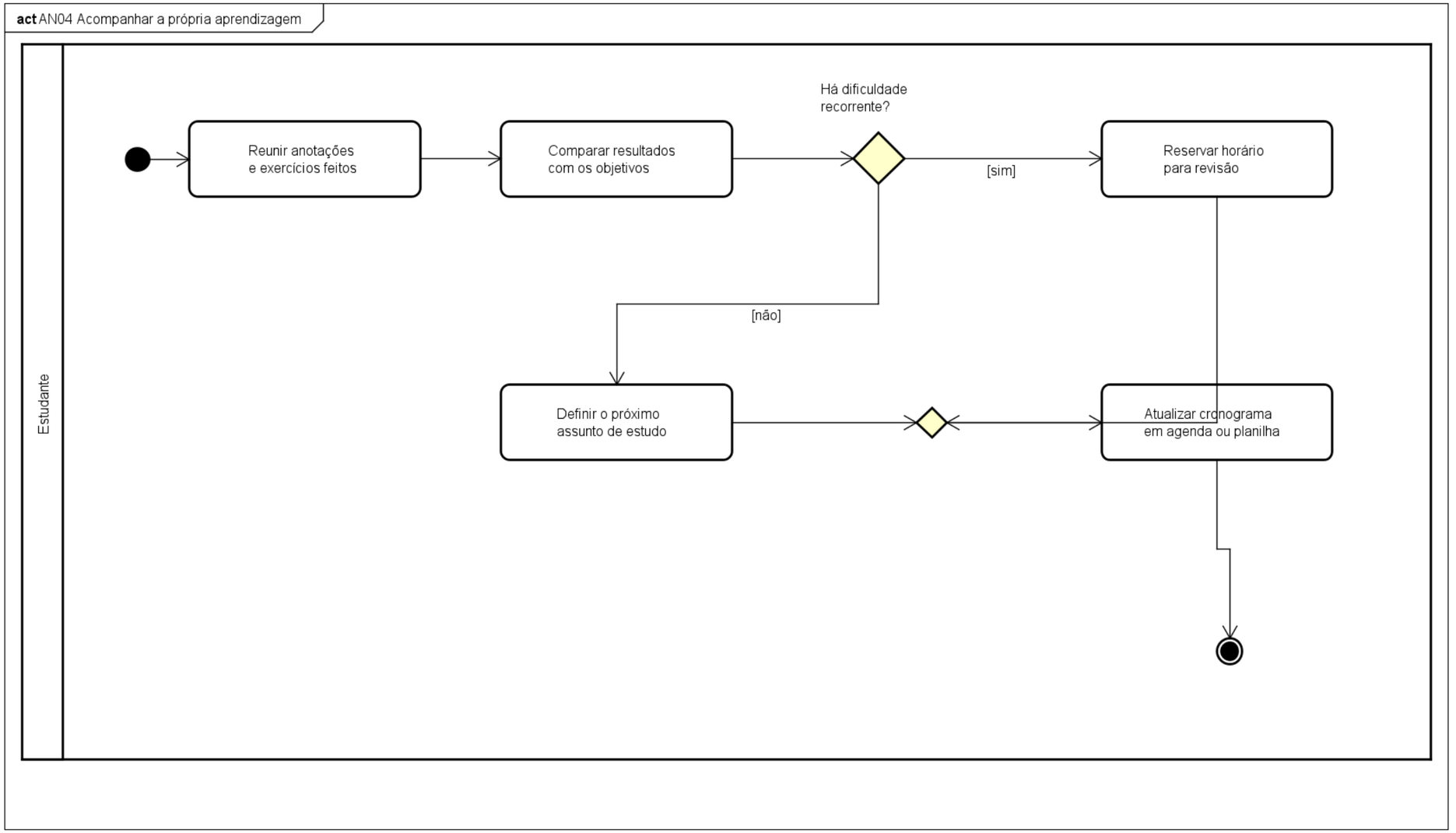
Praticar exercícios de programação

AN03 Processo atual e responsabilidades



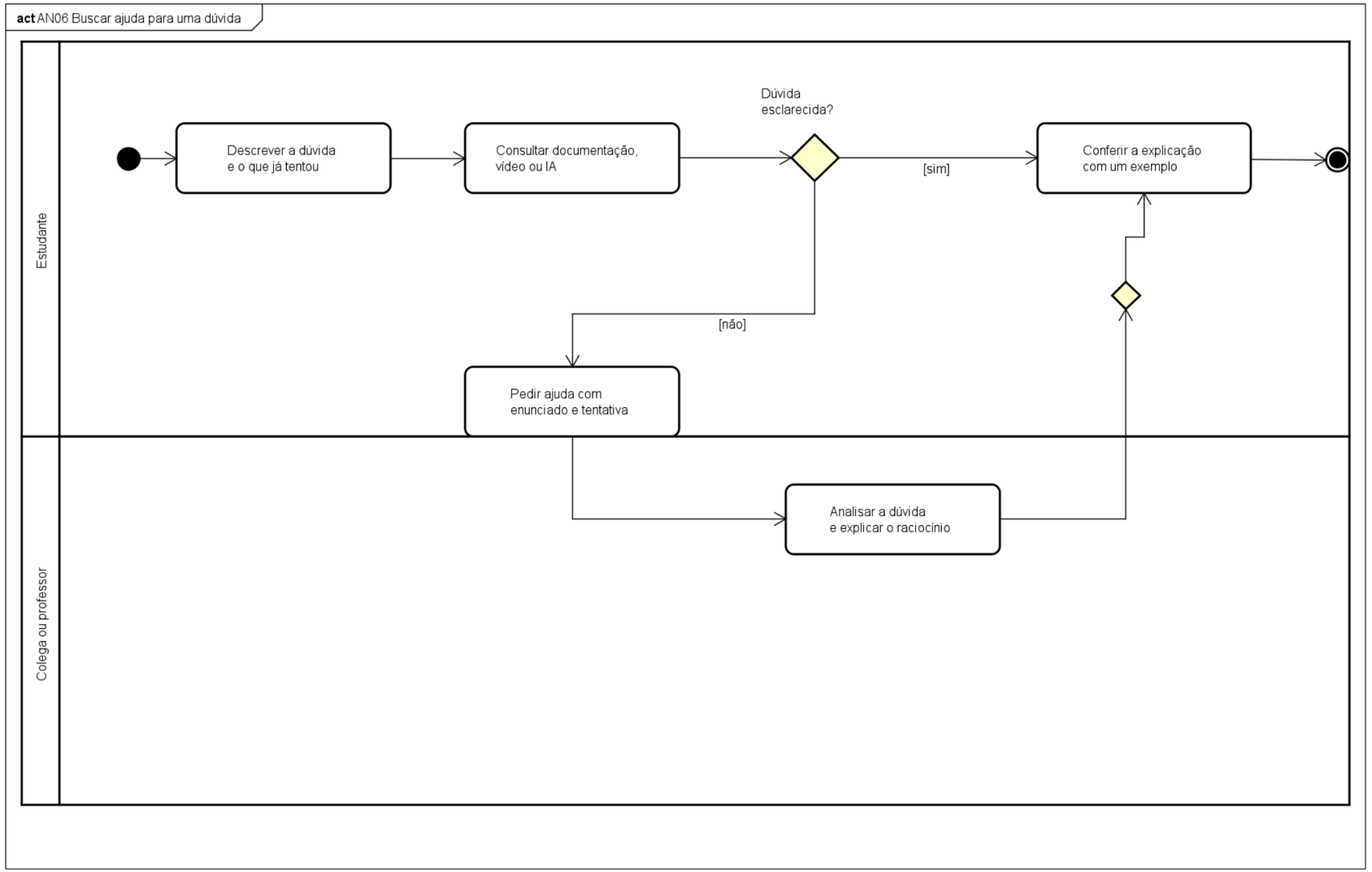
Acompanhar a própria aprendizagem

AN04 Processo atual e responsabilidades



Buscar ajuda para uma dúvida

AN06 Processo atual e responsabilidades



Referências

Orientações da Profª Patrícia

Revisão de 28 de setembro de 2026, consolidada no documento fornecido.

Gilleanes T. A. Guedes

UML 2 Uma abordagem prática. Classes, decisões e partições de atividade.

Código e modelos do Stacklyst

Commit 9256a4f, schema Prisma e projeto Stacklyst-seminario.asta.